A Message Cannot Arrive Before It Is Sent

Physical-Consistency Auditing for Streaming Latency Benchmarks, and What It Left of a Kafka-versus-Redis Comparison

GUSTAVO PEDRO RICOU, School of Computer Science and Statistics, Trinity College Dublin, Ireland

We set out to answer an ordinary question: for a real-time sports data feed, does the choice between Apache Kafka and Redis Streams affect end-to-end delay, and how does that change with the number of concurrent feeds? We built the benchmark, drove it with 3,315 real football matches on their recorded event schedule, and obtained a clean answer — Redis broker delay rising monotonically with concurrency while Kafka stayed flat, $p = 9.0 \times 10^{-11}$, no overlap between the two systems' run distributions, exactly as a single-threaded server should behave.

It was an artefact. Broker delay subtracts a timestamp taken in the producer process from one taken in the consumer process, so it admits a check no statistic supplies: the result cannot be negative. Applying that check to every run, rather than to the runs whose results looked wrong, rejected 1,321 of 2,266 runs, including every run behind the finding above. The rejected data is invisible to conventional inspection: medians stay positive, intervals stay narrow, effect sizes stay large, and the direction agrees with theory.

We report the check as a stated rule with its sensitivity analysis, and re-run the study inside it. What survives is narrower than what we set out to find. The two brokers are statistically equivalent within 1 ms and neither degrades across the concurrency range, a conclusion we show is robust to the unequal retention the check itself introduces, though a powered replication at a verified rate resolves what the original corpus could not: a small, tight, reproducible 0.41 ms transport advantage for Redis, well inside the millisecond margin but not the statistical tie the first measurement suggested. A second headline — a twentyfold end-to-end gap between the systems — also failed to reproduce, and we withdraw it: the runs behind it matched a median of seven events each, so a one-off producer start-up cost was being reported as a per-event constant. A sweep over the observation window settles which it is, by counting rather than averaging: lengthening the run multiplies its events several-fold while the number of affected events stays at four. That second withdrawal is instructive precisely because the integrity check does not catch it; a benchmark can be causally consistent, gate-clean, and still measure its own start-up.

Four falsifiable rules derived from a model of the failure are then tested directly. Inversion probability falls as the measured quantity grows ($\rho_s = -0.80$), rises with concurrent process count ($\rho_s = +0.80$), and follows the $\rho/(1-\rho)$ waiting time of an M/G/1 queue in utilisation ($\rho_s = 0.98$; $R^2 = 0.95$ against 0.64 for a linear alternative); and a symmetric instrument shrinks the residual between-system difference by a quarter, entirely on the side whose stamp was asymmetric. The practical consequence is that the check binds hardest exactly where a benchmark's measured effect is smallest — which is to say on the delicate comparisons such papers exist to make.

CCS Concepts: • **Computer systems organization → Distributed architectures**; • **Software and its engineering → Software performance**.

Additional Key Words and Phrases: streaming systems, latency benchmarking, measurement validity, Apache Kafka, Redis Streams, reproducibility

1 Introduction

Message brokers carry event data between producers and consumers in most real-time analytics pipelines, and two products dominate the role. Apache Kafka [12] stores messages in a partitioned, replicated log written to disk, designed for durability and high throughput. Redis Streams [22] keeps them in memory in an append-only structure, designed for minimum delay per operation. Practitioners choose between them largely on grey-literature comparisons that report Redis as

Author's Contact Information: Gustavo Pedro Ricou, School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland, pedrorig@tcd.ie.

several times faster [4, 10, 31], none of which is reproducible and none of which uses a workload from the domain the reader works in.

1.1 The original question

This work began as a straightforward attempt to fix that, for one domain:

> *Using the publicly available StatsBomb open dataset (2003–2023) and open-source load-generation scripts, compare end-to-end lag between Redis Streams and Apache Kafka for real-time sports data feeds, under varying concurrency.*

The domain supplied something a synthetic benchmark cannot: a real arrival process, and a real answer to *how many feeds run at once.* Football match records carry kick-off times, so the number of simultaneous feeds is an empirical quantity rather than a swept parameter. We treat that workload throughout as the *setting* that produced our findings rather than as a contribution in itself, and Section 3 describes it only to the depth needed to interpret the results.

1.2 The answer we obtained first

Our first campaign returned a clean result. Replaying distinct real matches across one, five and ten concurrent feeds, Redis broker delay rose monotonically (1.006 $\rightarrow$ 1.197 $\rightarrow$ 1.346 ms) while Kafka stayed flat to within 0.3%. Every comparison was significant after Holm correction [8], the strongest at $p = 9.0 \times 10^{-11}$, and at five feeds and above the two systems' run distributions did not overlap at all.

The finding was not merely significant; it was explicable. A Redis server executes commands on a single thread, so concurrent streams contend for one execution context, whereas Kafka's partitioning admits genuine parallelism. The measurement appeared to reproduce the textbook architectural distinction between the two designs, in the predicted direction and at the predicted scale.

It was an artefact. Broker delay is computed as consumer receipt minus broker acknowledgement, two timestamps taken in two processes. A message cannot arrive before it is sent, so a negative value is not measurement noise but proof that the instrument has failed. Checking every run against that constraint — rather than only the runs whose results looked implausible — rejected 862 of 1,382 single-machine runs and 459 of 884 multi-machine runs, including every run behind the result above.

1.3 Why the failure is worth a paper

Timestamp inversion under load is invisible to the checks a reviewer would apply. Medians stay positive and plausible. Confidence intervals stay narrow. Effect sizes are large and the direction agrees with theory. Only 5 to 14% of individual events invert, which is enough to displace a median by a fraction of a millisecond in a measurement whose entire effect is a fraction of a millisecond, and far too little to make any aggregate look wrong.

A benchmark can therefore produce a complete, internally consistent and entirely false result set. No amount of statistical care exposes it, because the contamination is systematic rather than random: greater care produces tighter intervals around the wrong number. Only a check against a physical constraint exposes it, and it must be applied to every condition as a rule.

1.4 Contributions

(1) **A physical-consistency audit for cross-process latency benchmarks** (Section 4.3), stated as a rule, applied uniformly to all 2,266 runs, with its threshold sensitivity reported rather than asserted (Section 6). We show the audit binds hardest exactly where the measured

effect is smallest, which inverts the usual intuition that large clean effects are the trustworthy ones.

(2) **A demonstration**, which we regard as the strongest of the four: a complete, significant, theory-confirming result set that was physically impossible, reported in full so a reader can judge how convincing invalid data looks (Section 5).

(3) **Four falsifiable rules, derived and measured** (Sections 4.4 and 7.3). From a model of the failure we derive that inversion probability is $F_\Delta(-T_{\text{true}})$ and test its consequences: it falls as the measured quantity grows, rises with concurrent process count, follows M/G/1 waiting time in utilisation, and — the rule that bears directly on our own first result — an asymmetric instrument leaves a between-system difference that a symmetric one removes. All four hold. This converts the central property from an observation into a measured law.

(4) **A second withdrawal, and a test that would have caught it** (Section 7.2): a twentyfold end-to-end gap we had reported, and built a recommendation on, turned out to be a per-run start-up cost read as a per-event constant, because the runs behind it matched a median of seven events. We give a controlled discriminator — sweep the observation window and count the affected events rather than averaging them, since a per-run cost holds that count fixed while a per-event one grows it — and trace the cost to a single blocking first `produce()` and the four events that queue behind it. The integrity check does *not* catch this, which is the point: causal consistency is necessary and not sufficient, and a percentile over single-digit samples describes the harness rather than the system.

(5) **A configuration result** (Section 7.5): each system has exactly one client-side setting worth one to two orders of magnitude in delay, and both are free on a co-located testbed.

2 Background and Related Work

2.1 Streaming benchmarks

Stream processing has a mature benchmark literature. Linear Road [1] is the canonical application benchmark; NEXMark [29] models an online auction and has become the standard query suite for dataflow engines; ESPBench [6] constructs an enterprise workload; RIoTBench [26] supplies IoT dataflows; and the OpenMessaging Benchmark [20] measures broker throughput and delay directly across messaging systems. Stonebraker et al. [28] set the requirements these evaluate against.

Two properties of this body of work matter here. Each drives its system with a synthetic arrival process chosen for analytical tractability. And none, to our knowledge, reports what fraction of its runs it discarded on integrity grounds.

There is precedent for the kind of result we report. Schroeder et al. [23] showed that whether a load generator is open or closed — a modelling choice most benchmarks make implicitly — changes measured behaviour so much that results obtained under one model do not transfer to the other. Their contribution was not a new system but a demonstration that a widespread methodological choice silently determined published conclusions. Ours is of that shape: an unexamined property of how benchmarks record time, shown to determine the result. Mohammad [19] surveys benchmark practice in Kafka event-streaming systems and finds reported results frequently not comparable, because client configuration, acknowledgement semantics and instrumentation differ between the systems under test. Our experience supports that critique and extends it: comparable configuration is not sufficient, because the instrument itself can fail silently under load.

2.2 Measuring time across processes

Lamport [15] established that a distributed system has no single physical time, only a partial order induced by causality. Our check is the simplest application of that principle: a broker's acknowledgement causally precedes the consumer's receipt, so a measurement in which receipt is timestamped earlier violates the causal order and supports no conclusion.

The engineering problem is well known and partially solved. Systems approximate physical time with synchronisation protocols, of which NTP [18] is the most widely deployed; Spanner [3] goes further and exposes clock *uncertainty* to the application, on the grounds that a system which pretends to know the time exactly will eventually be wrong undetectably. Closest to our setting, Cloudprofiler [30] addresses precisely the inadequacy we encountered — that millisecond-grade synchronisation is too coarse to profile modern streaming engines — by calibrating time-stamp counters across nodes during quiescent periods, reaching roughly 92 μs accuracy.

We therefore make a narrower claim than "nobody has considered this". The accuracy problem is recognised and better instruments exist. What appears to be missing is the *practice*: an audit applied uniformly to every run of a campaign, with the rejection rate reported the way a survey reports its response rate. Cloudprofiler improves the instrument; we argue that whatever instrument is used, its output should be checked against causality and the check should be published. Our own failure is also not one of clock synchronisation — both processes read the same operating-system clock on one machine — but of *when the timestamp is taken*: a thread descheduled between the event and the call recording it stamps a time later than the event, and under CPU saturation that delay routinely exceeds the interval being measured.

Two 2026 results sit close to ours and we are explicit about what each already establishes. Chandrasekar and Kramberger [2] identify a systemic client-side measurement bias in production LLM inference benchmarks — a single-process client that cannot keep up under concurrency and inflates the latencies it reports — and model it as an M/G/1 queue, a framing we adopt in Section 4.4. Sharma et al. [25] go further in our direction: they report *negative timing spans* in a multi-node inference pipeline, including one built on Kafka, and show that causality violations emerge from a few milliseconds of clock skew while the system continues to function correctly.

We therefore do not claim that causality-violating measurements are a new observation. Three things remain open after that work, and they are what this paper supplies. First, Sharma et al. *inject* clock skew between nodes; our violations arise with no skew at all, on a single machine where both processes read the same clock, purely from scheduler delay in reaching the clock. That is a harder failure to defend against, because synchronising clocks does not remove it. Second, neither paper states a detection rule, applies it uniformly, or reports what fraction of a campaign it rejects. Third, and most importantly, neither relates violation probability to the magnitude of the quantity being measured. Section 4.4 derives that relation and Section 6.6 shows what follows from it: the check is least forgiving exactly where a benchmark is most delicate.

One further difference decides how each failure can be caught at all. A load-induced inflation is wrong but physically possible, so detecting it needs a better instrument or an independent reference. A causality violation is self-detecting, at no cost and with no reference. That property, rather than the existence of load-induced bias, is what we build on.

Jain [9] sets out the standard requirement that an instrument be validated on the system it will measure. The check in Section 4.3 is a cheap, specific instance of that for cross-process delay.

2.3 Statistical methods

Latency distributions are non-normal, so difference tests are rank-based: Mann–Whitney U [17] per condition and Kruskal–Wallis [13] across concurrency levels, with Holm's correction [8]. Because several claims here are negative, we state them with two one-sided tests [14, 24] against a pre-specified margin rather than by failing to reject equality. As a point estimate of the difference between two systems we use the Hodges–Lehmann shift [7] with percentile bootstrap intervals [5];

Table 1. The workload, from 3,315 matches. Medians across matches; peak rate is the maximum over any sliding ten-second window.

Property	Value
Events per match	3,507
Match duration	8,412 s
Mean arrival rate	0.415 ev/s
Peak rate (10 s window)	2.70 ev/s
Peak-to-mean ratio	6.45
Median inter-arrival gap	1.0 s
Distinct kick-off instants	2,346
Max simultaneous kick-offs	12
Max matches in play at once	21

Section 6.4 shows that this estimator's breakdown point is what makes our main conclusion robust to the audit's own side effects.

3 The Setting

The workload is a live football event feed, and we describe it only as far as the results require. Full characterisation of 3,315 matches from the StatsBomb open dataset [27], spanning 52 competition-seasons from 2003 to 2023, is available in the artefact; event data of this kind is collected by trained human operators, whose accuracy has been studied [16, 21].

Three properties of the workload drive every experimental choice (Table 1).

It is sparse. A match produces a median of 3,507 events over roughly 8,412 seconds, a mean rate of **0.415** events per second. This is four orders of magnitude below the rate at which either broker begins to strain, and it is the property that makes Section 7.2's result possible.

It is bursty. Over a sliding ten-second window the same feeds peak at 2.70 events per second, a peak-to-mean ratio of 6.45. Half of all inter-arrival gaps are one second or shorter. The feed is idle most of the time and busy in short bursts, which is the duty cycle under which a client that must wake from idle pays that cost on nearly every event.

Its concurrency is bounded and knowable. Match records carry kick-off times, so the number of simultaneous feeds is empirical rather than swept. Across 2,346 kick-off instants the largest carries **12** simultaneous matches, and at most **21** are in play at once. Domestic leagues schedule the final matchday simultaneously by rule, so a 20-team league plays ten concurrent matches. We therefore benchmark at $N \in \{1, 9, 10, 12\}$: the median slot, the upper quartile, the structural league bound, and the observed maximum. Two limits apply. The dataset is a sample, so observed simultaneity is a lower bound; and the bound is per league, so a platform serving many competitions faces their sum.

4 Method

4.1 Testbeds

Because several of our earlier numbers proved to be properties of a machine rather than of either broker, we state both testbeds and measure the floors that bound resolution.

Testbed A (single machine). Windows 11, 8-core AMD Ryzen, 31 GB RAM, CPython 3.9. Producer and consumer are native processes; Kafka 4.1.1 and Redis 7.2.4 run in Docker containers whose engine executes in a WSL2 virtual machine, reached through published ports. Two floors bound it:

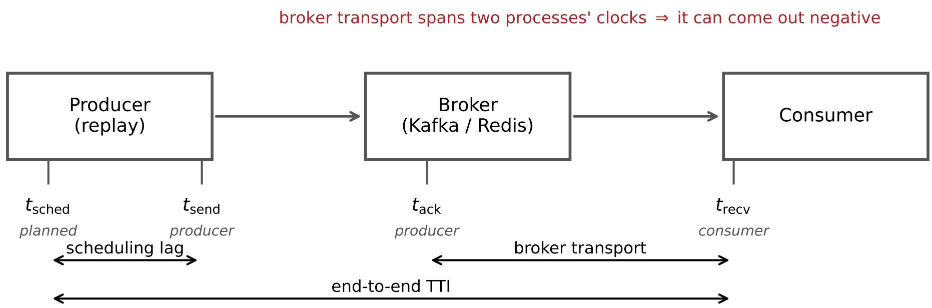

Fig. 1. The four timestamps and the intervals each measure spans. Scheduling lag is measured within one process. Broker transport subtracts a producer-side timestamp from a consumer-side one, and can therefore come out negative when either process is delayed.

a requested 1 ms sleep takes a median of 15.6 ms (the Windows timer tick), and opening a TCP connection to a published port costs 5.6–9.9 ms.

Testbed B (four machines). Four Oracle Cloud `VM.Standard.E5.Flex` instances — three brokers and one driver — on Ubuntu 22.04, CPython 3.10, over a private network, with client libraries pinned identically to Testbed A. Both floors disappear: a 1 ms sleep takes 1.06 ms, and a broker round trip on an established connection is 0.22 ms locally and 0.24 ms across machines. A genuine two-machine network is thus *faster* than Testbed A's co-located path, which is the sharpest statement we can offer of how much a testbed distorts a benchmark. Every result reported as a finding comes from Testbed B.

4.2 Measurements

Each match is preprocessed into a replay plan recording, per event, the time at which it should be emitted. A producer replays a plan at a configurable speed-up; a consumer records arrivals. In the concurrency experiment each of the N feeds carries a *distinct* real match at its true rate: an earlier design gave every feed the same merged ten-match plan, which made the concurrency axis covertly a throughput axis.

The primary measure is **Time-to-Insight** (TTI), from an event's scheduled emission to its availability at the consumer. We decompose it (Figure 1):

$$\text{TTI} = \underbrace{(t_{\text{send}} - t_{\text{sched}})}_{\text{scheduling lag}} + \underbrace{(t_{\text{recv}} - t_{\text{ack}})}_{\text{broker transport}} + \text{residual}.$$

Scheduling lag is measured entirely inside the producer and characterises the client. Transport spans the broker and characterises the product. The distinction is not cosmetic: for this workload the two have opposite answers, and reporting only the aggregate is what concealed the result in Section 7.2.

Critically, t_{ack} is stamped in the *producer* process (from Kafka's send callback, or on return from Redis's `XADD`) while t_{recv} is stamped in the *consumer*. That subtraction is the one that can go wrong.

4.3 The consistency check

We implement the causal constraint as a stated rule, fixed before the final campaign and applied to every run including those whose results looked reasonable. A run is **rejected** if either

(1) more than 1% of its events carry a negative value in any latency component, or
(2) the median of any component is negative.

A condition is usable only if all of its runs survive. Where a condition is partly rejected we report the retention rate, and Section 6.4 bounds the selection this introduces. The tool exits non-zero when any run fails, so a campaign script can stop on it.

4.4 A model of the failure, and what it predicts

A rule that rejects runs is only useful if one can say *when* it will fire. This subsection states the model that makes that prediction, because it is what converts the check from a hygiene step into something a reader can apply to their own measurement without repeating our experiment.

Every timestamp is taken by software some interval after the physical event it claims to mark, because the thread that reads the clock must first be scheduled. Writing those stamping delays as δ_{ack} and δ_{recv},

$$T_{\text{measured}} = T_{\text{true}} + \Delta, \qquad \Delta \equiv \delta_{\text{recv}} - \delta_{\text{ack}}, \tag{1}$$

so an inversion occurs exactly when $\Delta < -T_{\text{true}}$, and

$$\Pr[\text{inversion}] = F_{\Delta}(-T_{\text{true}}). \tag{2}$$

Figure 2 draws both halves of this. Panel (a) shows the mechanism: the acknowledgement is stamped late enough that the consumer's receipt timestamp precedes it, even though the message plainly arrived after it was sent. Panel (b) shows the consequence that matters for benchmark design.

Equation 2 says something that is easy to miss and, we think, the most useful thing in this paper. **Inversion probability is a property of the quantity being measured, not only of the machine.** The same instrument, on the same host, under the same load, is more likely to be invalid when measuring a small latency than a large one, because a fixed distribution of Δ overlaps a small T_{true} almost entirely and a large one hardly at all. This is why our network-delay arm, where transport is tens of seconds, passes the check on every run while same-hardware arms measuring one millisecond fail wholesale.

Three further predictions follow — on utilisation, on process count, and on instrument asymmetry — and Section 7.3 reports the experiments that test them.

Utilisation, and what drives it. δ is the waiting time of a runnable thread. Treating the CPU as a server, the Pollaczek–Khinchine result for an M/G/1 queue gives mean waiting time growing as $\rho/(1-\rho)$ in utilisation ρ. The spread of Δ should therefore be small and flat at low utilisation and grow sharply near saturation: a *knee*, not a linear ramp. This is the same framing Chandrasekar and Kramberger [2] apply to client-side bias in inference benchmarks. A corollary sharpens it into a second prediction: ρ is set by the number of *runnable threads* per core, not by the offered event rate, so at a *fixed* aggregate rate, adding concurrent producer/consumer processes should raise inversion rate — the failure follows process count, not load.

Asymmetry. It is $\mathrm{E}[\Delta]$, not its variance, that biases a *comparison.* Symmetric instrumentation gives $\mathrm{E}[\Delta] \approx 0$: noise, no systematic error. Asymmetric instrumentation does not — and if two systems under comparison stamp differently, the instrument manufactures a difference between them. This is precisely our situation: Kafka's producer stamps the acknowledgement in an asynchronous callback, Redis's on return from a blocking command. It is the most plausible account of how our first result set acquired a large, significant, entirely spurious effect.

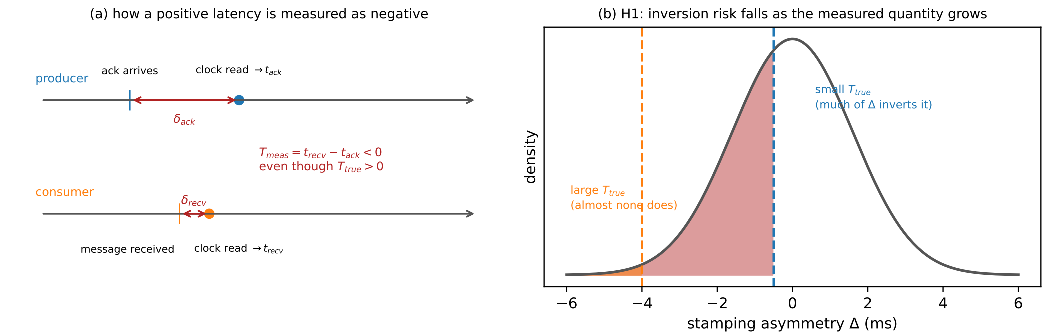

Fig. 2. (a) How a positive latency is measured as negative: each side stamps its clock some interval after the event, and if the producer's delay exceeds the true transport plus the consumer's delay, the difference inverts. (b) Why the check bites hardest on small effects. The same distribution of stamping asymmetry Δ inverts most of a small T_{true} and almost none of a large one.

4.5 What the check cannot catch

A paper about measurement validity should be explicit about the limits of the instrument it proposes, so we state them before reporting any result with it.

Equation 2 makes the blind spot precise. The check fires only when $\Delta < -T_{\text{true}}$: it detects the tail of the asymmetry distribution that crosses zero, and nothing else. Three consequences follow, and the third is the serious one.

It cannot see bias smaller than the measurement. If Δ displaces every measurement by, say, 0.3 ms while T_{true} is 1 ms, no value goes negative and the check passes a corpus whose every number is 30% wrong. Passing is therefore evidence that the instrument did not fail *catastrophically*, not evidence that it was accurate. We use the word "sound" in this narrow sense throughout.

It says nothing about the tail it does not reach. A run just above the threshold and a run just below it may differ arbitrarily in bias. The rejection rate is a hygiene statistic, not an error bar.

It is blind to symmetric inflation. The failure Chandrasekar and Kramberger [2] describe — a saturated client inflating every latency it reports — keeps all measurements positive and passes our check completely. Their failure mode and ours are complementary, and neither instrument detects the other. A benchmark that applies only the check proposed here is still exposed to theirs.

This is why we frame the contribution as a *necessary* condition rather than a validation procedure. A corpus that fails the check is unusable. A corpus that passes it has cleared the cheapest available bar and nothing more.

4.6 Statistics

Margins are fixed before testing and proportionate to the quantity compared: $\delta = 1$ ms for broker transport, a quantity of order one millisecond, and $\delta = 40$ ms for end-to-end TTI, one video frame at 25 fps. Holm families are declared over the design actually executed: the four per-N transport comparisons form one family, the network-delay arm another; omnibus Kruskal–Wallis tests are uncorrected; equivalence tests are pre-specified and belong to no family. Surviving samples range from 8 to 35 runs per cell; the 8-run cell is treated as descriptive and flagged wherever it appears.

Table 2. **The first result set, later rejected.** Broker transport (median, ms) by concurrency, Testbed A, 10× replay. Every cell fails the check of Section 4.3.

N	Kafka	Redis	Difference	p_{Holm}
1	0.999	1.006	0.007	1.0×10^{-4}
5	1.001	1.197	0.196	6.5×10^{-6}
10	1.002	1.346	0.344	9.0×10^{-11}

5 The Answer We Obtained First

We present the first result set in full rather than describing it, because a reader cannot otherwise judge how convincing an invalid corpus looks.

5.1 The result

On Testbed A, replaying distinct real matches at ten times real speed across one, five and ten feeds with 15–30 runs per cell and both producers pipelined, broker transport separated the two systems cleanly (Table 2). Every comparison was significant after correction and at $N \geq 5$ the run distributions did not overlap.

5.2 The checks it had already passed

The corpus was not naive. It was the product of five earlier corrections, each of which had reversed the apparent winner and each found by investigating a result that looked wrong:

(1) *A process-relative clock.* Cross-process timestamps taken with `perf_counter_ns`, whose origin is per-process, injected each run's consumer start-up offset into every measurement. Fixed with a shared wall-clock epoch.
(2) *A saturating replay speed.* At 120× the producer could not keep pace, inflating scheduling lag to tens of seconds.
(3) *An asymmetric load generator.* The Kafka producer blocked on a broker round trip per event while the Redis producer dispatched asynchronously. Median TTI at one feed fell from 1,644 ms to 16 ms once the Kafka producer was pipelined.
(4) *A concurrency axis that was covertly a throughput axis*, from the merged-plan design described in Section 4.2.
(5) *A heavy tail contaminating every mean.* Kafka's end-to-end 95th percentile sat at 99–105 ms at every concurrency level while its median moved from 1.8 to 7.2 ms. A 95th percentile that ignores load is not system behaviour; decomposition located it in scheduling lag, not transport.

We also verified the comparison was not biased by message size or protocol: payloads are near-identical (median ≈170 bytes) and serialisation costs are negligible and comparable.

A sixth problem shaped our later protocol. Our first run of the acknowledgement experiment of Section 7.4 reported no improvement whatsoever — a clean refutation of our own hypothesis. The treatment had never reached the consumer: the orchestration script accepted the option but, through a failed edit, never passed it on. We found this only by deliberately supplying an invalid option and observing that the consumer did not reject it. A null produced by an unapplied treatment is indistinguishable in the data from a genuine one, so every intervention reported here carries a manipulation check that aborts the campaign if the treatment is not accepted.

Table 3. Consistency audit of every run in the study.

Corpus	Runs	Rejected	Conditions	Usable
A (single machine)	1,382	862 (62.4%)	76	8
B (four machines)	884	459 (51.9%)	40	13
Total	2,266	1,321 (58.3%)	116	21

The point of the list is not the corrections. It is that a corpus which had survived six of them, a fairness audit, and a full battery of rank tests, effect sizes and multiplicity correction, was still entirely invalid, and nothing in its output said so.

6 The Audit

6.1 What it rejects

Applying the rule to all 2,266 runs rejects 1,321 of them (Table 3). On Testbed A only 8 of 76 conditions survive intact; on Testbed B, 13 of 40. No condition behind Table 2 survives.

6.2 Why it fails, and why it looked correct

The cause is legible in *which* conditions fail: rejection tracks replay acceleration, with the accelerated concurrency conditions failing and the unaccelerated ones passing. We state that as a direction rather than as two rates, because the rate is one thing our own artefacts do not record — see Section 6.5. The mechanism is the one described in Section 2.2. The acknowledgement timestamp is taken by the producer's callback thread; when accelerated replay saturates the driver's CPU that thread is descheduled, and by the time it reads the clock the consumer has already recorded receipt. It is a timestamping race under load, not clock skew, which is why medians stay positive at moderate load and invert wholesale only at saturation. At one hundred connections the failure becomes gross enough to see unaided: Kafka's median transport is reported as −6.4 ms.

The corpus looked healthy for an arithmetic reason. Between 5 and 14% of events invert, which displaces a median by a few tenths of a millisecond in a measurement whose entire effect is 0.34 ms. The bias is also *asymmetric* between arms, because the two clients record the acknowledgement differently — Kafka from an asynchronous callback, Redis on return from a blocking command — and an asymmetric bias between two arms of a comparison manufactures a difference where none exists.

6.3 How much the threshold matters

We expected the distribution of inversion rates to be clearly bimodal, which would have made the 1% threshold uncontroversial. It is not (Figure 3a): only 104 of 1,382 runs are completely clean and the rest spread over three orders of magnitude with no natural gap. The threshold is a real decision, so we report its consequences rather than asserting robustness.

It strongly affects *how many runs* are rejected — from 92.5% at zero tolerance to 19.2% at a permissive 20% (Figure 3b). It does not affect *which conditions are usable*, and therefore changes no conclusion here: a condition requires all its runs to survive, and even at a 5% threshold the accelerated arms retain only 32–77% of theirs. The withdrawal of Table 2 holds across the entire range.

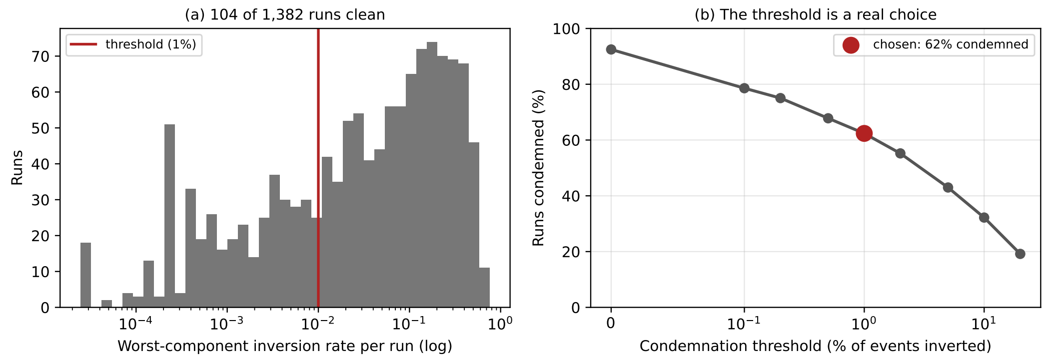

Fig. 3. The audit on Testbed A. (a) Per-run inversion rates: 104 runs are clean and the rest spread over three decades, so there is no natural threshold. (b) Share of runs rejected against the threshold. The choice matters for the run count and not for any conclusion.

Table 4. Bounding the unequal-retention selection effect. "Worst case" imputes every rejected Redis run at the largest value observed in that cell. Margin 1 ms.

N	Redis retained	Shift (ms)	Worst case	Equivalent
1	8/8	+0.081	+0.081	yes
9	20/27	+0.021	−0.051	yes
10	17/30	+0.116	−0.485	yes
12	19/36	+0.053	−0.227	yes

6.4 Bounding the audit's own selection effect

An audit that rejects unequally between arms introduces selection. In our concurrency experiment Kafka passes on 100% of runs at every level while Redis passes on 53–100%. Since inversion is caused by driver saturation, the surviving Redis runs are plausibly the less-loaded ones, and the bias would run in exactly the direction that manufactures our reported equivalence.

The rejected runs' values are not recoverable, so a threshold sweep is impossible. Instead we bound: impute every rejected Redis run at a hypothetical value and ask how large it must be before the conclusion changes (Table 4). Equivalence survives at every level, and no imputed value up to 1 second breaks it.

The reason is structural, and so is its limit. The Hodges–Lehmann shift is a median of pairwise differences and has a breakdown point of one half: while more than half the runs survive, nothing the rejected runs could have contained moves the estimate outside the margin. Our tightest cell is N=12 at 52.8% retention — only 2.8 points above breakdown. Had retention fallen below one half this defence would not apply, and we would have had to withdraw the cell.

6.5 A gap in our own provenance

Auditing the corpus turned the same scrutiny on our own record-keeping, and it did not survive it either. We report the finding here because the protocol in Section 8.2 is weaker without it.

Replay plans built from StatsBomb carry a **baked-in** 120× **time compression**: a plan's emission offset is already the match time divided by 120. The producer's `-speedup` flag then multiplies that. So `-speedup 1` against such a plan is *not* real time, it is 120×; true real time needs 1/120. The same flag means different things depending on which plan it is pointed at, and getting it wrong is silent: the run completes, the wall time looks unremarkable, and the numbers stay plausible. We lost a campaign to it before adding a pre-flight that derives the value from the plan and then checks the achieved rate against elapsed wall time.

The part we cannot repair is retrospective. **No surviving artefact records the achieved replay rate of any run we report.** Several superseded Testbed A scenarios record the nominal `-speedup` *flag*, which is not the same thing: converting a flag to a rate requires the plan's compression, and every plan those files name has since been deleted. Everywhere else the orchestrator wrote `speedup` into a per-campaign summary that lived beside the raw per-run data, and for the earliest corpora only the aggregated CSVs were kept. For the E1 corpus of Section 7.1 the surviving evidence is contradictory: the commit that introduced it states the runs were made at true real time, the campaign script reconstructed afterwards specifies `-speedup 10`, and the flag's meaning was corrected 21 hours later the same day. E1 was replayed at 1×, 120× or 1200×, and we cannot say which.

The rate is recoverable from the measurements, if not from the metadata. Having argued all paper that a physical constraint can decide what record-keeping cannot, we applied the same move here, and it works. The start-up cost of Section 7.2 is a clock: it lasts a fixed 102.6 ms of *wall* time, so the number of events it swallows depends directly on the replay rate. Counting how many events per run carry it therefore reads the rate back out of the data.

The E1 runs sort into cells by how many events they matched, and one cell is diagnostic. Fifteen Kafka runs matched exactly eight events, and their median scheduling lag is **52.34** ms (range 51.60–53.01) — not 103 ms, and not 1.6 ms. A median of eight values is the mean of the fourth and fifth. The only way to land midway between the two modes is for exactly four of the eight to be late, so the fourth value is a fast event and the fifth a slow one: $(1.59 + 103.5)/2 = 52.6$ ms against 52.34 observed. Four is also exactly what the loop trace shows at a verified real-time rate (Table 7), and it is what the plan predicts, since five events share $t_{\mathrm{sim}}{=}0$ and the first of them pays the block rather than inheriting it.

That cell excludes the accelerated alternatives outright. At 120× the prologue spans 12.3 s of match time and holds 16 events; at 1200×, 123 s and 112 events. Under either, all eight matched events would be inside it and the cell would read 103 ms. It reads half that. **E1 was replayed at true real time**, the commit record was right, and the campaign script reconstructed afterwards describes a different, earlier campaign. The remaining cells agree: runs matching five or seven events read 103 ms, as four late events out of five or seven must give.

We leave the disclosure standing rather than deleting it, for two reasons. The recovery was available only because the artefact we were chasing happened to be a wall-clock constant with a sharp signature; a reader should not have to reconstruct an experimental parameter by arithmetic on its results, and next time there may be nothing to reconstruct it from. And the exercise makes the protocol rule concrete: what was missing was one number, written down once.

What the gap touched. The audit is unaffected either way: a negative transport is causally impossible at any replay rate, and the rejection counts are arithmetic on surviving per-event data. The withdrawal in Section 7.2 is unaffected by construction — we give the argument in a form that holds at every candidate rate. The rules of Section 7.3 were measured after the pre-flight existed, at a derived and verified rate. Section 7.1's external validity is restored by the recovery above, though we now state it as an inference from the data rather than as a property of the campaign we can document. Section 8.2 carries the rule that would have made all of this unnecessary.

6.6 The property that makes this general

One feature transfers beyond this study. The check tests against zero, so it rejects a measurement in proportion to how close that measurement sits to zero. Our network-delay arm, where Redis transport is tens of *seconds*, passes at 15 runs out of 15 at every injected delay — on the same hardware, in the same campaign, minutes apart from conditions that fail outright. A few milliseconds of timestamping race is invisible against a 31-second effect and fatal against a 0.34-millisecond one.

The check therefore bites hardest exactly where the scientific question is most delicate. This inverts a common intuition. In cross-process delay measurement, an effect of the same order as the instrument's own noise floor is the one most likely to have been manufactured by it, and statistical significance offers no protection because the artefact is systematic. We recommend that streaming benchmarks report an integrity audit as a matter of course.

6.7 What we withdraw

We withdraw the entire Testbed A arm, and on Testbed B the accelerated concurrency sweep, the connection sweep above ten connections, and the three-node cluster arm, which fails on every run in both systems.

7 What Survives

All results below come from Testbed B and pass the check. Each states its retention.

7.1 The brokers are equivalent within a millisecond, but not indistinguishable

The original E1 corpus, replaying distinct real matches at the concurrency the workload produces, retains 164 of 201 runs and reports broker transport flat across the range and near-equal between systems (Table 5); Kruskal–Wallis finds no significant concurrency effect in either backend ($p = 0.061$ for Kafka, $p = 0.091$ for Redis). Its replay rate is an inference from the runs rather than a documented setting, recovered by the argument in Section 6.5 as true real time. We could rest the equivalence claim on E1, but we do not, for a reason Section 7.2 makes unavoidable: those runs matched a *median of seven events each.* The transport comparison is computed over the same seven-event opening burst as the scheduling lag we withdraw, so it is both under-powered and drawn from the least representative part of the match. On seven events per run a sub-millisecond difference cannot be resolved, and E1 does not resolve one — its Hodges–Lehmann shifts wander (0.021, 0.116, 0.053 ms) and its N=1 estimators disagree. That E1 finds the two systems "equal" is as much a statement about seven events as about the brokers.

A powered replication. We therefore re-measured transport directly, at a replay rate derived from the plan and verified against wall time, over a median of 127 events per run rather than seven, at $N \in \{1, 9, 12\}$ with 15 replicates each (Table 6). The picture is sharper and it is not the one E1 painted. Broker transport is Kafka $\approx$ 0.54 ms against Redis $\approx$ 0.11 ms: a Hodges–Lehmann shift of 0.41 ms, Kafka slower, with a bootstrap 90% interval of width ±0.006 ms and $p < 10^{-26}$ at every N. The two systems are *not* statistically indistinguishable; Redis's in-memory `XADD` really is several times faster per operation than Kafka's replicated-log append, which is the direction the grey-literature comparisons report.

And yet the same three estimators — Welch, percentile bootstrap and Hodges–Lehmann — agree that the difference is *equivalent within one millisecond* at every N, because 0.41 ms is comfortably inside that margin. Both statements are true and neither is idle: against the seconds-scale human annotation latency that dominates any live sports feed's end-to-end budget, a 0.41 ms broker difference is parts in 100,000, so for choosing a broker it is noise; but it is a real, tight, reproducible effect that a properly powered instrument resolves and a seven-event one cannot. The shift is also

Table 5. End-to-end delay and its decomposition, Testbed B, after the consistency check (164 of 201 runs retained). Medians, ms. The replay rate is identical across both arms but not recorded in the surviving artefacts (Section 6.5); the scheduling-lag columns are withdrawn (Section 7.2) and retained only as evidence for that withdrawal.

	TTI		Sched. lag		Transport	
N	Kafka	Redis	Kafka	Redis	Kafka	Redis
1	105.3	5.0	103.0	1.4	0.792	0.721
9	105.5	5.4	103.0	1.9	0.802	0.807
10	105.3	5.8	102.8	2.0	1.000	0.855
12	105.7	5.3	102.9	1.7	0.839	0.844

Table 6. Powered transport replication at a verified true-real-time rate, over a median of 127 events per run (not the seven of Table 5), 15 replicates per cell. Medians in ms; the Hodges–Lehmann shift is $Kafka - Redis$ with a percentile-bootstrap 90% interval. The brokers are equivalent within 1 ms at every N (all three estimators) yet cleanly distinguishable: Redis is ≈ 0.41 ms faster, and the shift does not grow with concurrency.

N	Events/run	Kafka	Redis	HL shift [90% CI]
1	148	0.512	0.099	0.409 [0.394, 0.421]
9	121	0.539	0.115	0.418 [0.412, 0.424]
12	127	0.540	0.114	0.420 [0.414, 0.425]

flat across concurrency (0.409, 0.418, 0.420 ms at $N = 1, 9, 12$), so neither system degrades. A part of even this residual is the instrument rather than the broker: Section 7.3 shows that 0.07 ms of the gap is the asymmetric acknowledgement stamp, leaving a true broker-transport difference near 0.34 ms. That 0.07 ms is a concrete instance of the blind spot of Section 4.5: it is a bias smaller than the quantity measured, so it never turns a value negative and the consistency check passed both corpora that carried it. The check guarantees a corpus is not *catastrophically* wrong; resolving a 0.07 ms instrument artefact still took a controlled experiment.

This refines E1 rather than contradicting it, and it sharpens the contradiction with Table 2: the accelerated corpus reported Redis *degrading* with concurrency; the powered real-time measurement finds Redis uniformly *faster* and flat. The reversal is the audit's doing, and the measurement's.

7.2 The end-to-end difference, and why we withdraw it too

TTI and transport give opposite answers in Table 5. Of Kafka's 105.5 ms, 102.9 ms is producer scheduling lag against Redis's 1.8 ms, while broker transport is 0.84 and 0.80 ms. An earlier version of this paper reported that twentyfold end-to-end gap as a finding, attributed it to the client's sender thread waking from idle, and built a practitioner recommendation on it.

That gap does not survive re-measurement, and the reason is a second instance of this paper's own thesis. We report it at length for the same reason we reported the first.

The offset is a start-up cost, not a per-event constant. Re-measuring at N=1 on the same driver and the same broker, at a replay rate derived from the plan and verified against elapsed wall time, gives a median producer scheduling lag of 1.59 ms for Kafka, not 103 ms — with a *maximum* of 103.5 ms. Two independent instrumentation paths agree: the per-event loop trace and the per-run summary. The 103 ms is real, but it is paid once per run, not once per event.

We cannot call this a repetition of E1's condition, because E1's replay rate is not recoverable (Section 6.5). That is why the argument below does not rest on the comparison between the two campaigns at all. It rests on a sweep that is internally controlled — one campaign, one rate, one match, only the window varying — and on an arithmetic relation that holds at every rate E1 could have used.

A controlled test that separates the two. A median cannot distinguish a per-run cost from a per-event one, because how much of the cost the median sees depends on how many events the run contains — which is precisely what misled us. The quantity that *does* separate them is the *number* of affected events per run. A per-run cost predicts that number is fixed however long the run gets; a per-event constant predicts it grows with the run. We therefore replayed the same match at true real time under three observation windows, holding everything else fixed, and counted events waking more than 50 ms late (Table 7).

The count does not move. Going from the 60 s window to the 600 s window multiplies the events emitted by 8.9 — 57 to 507 — while the number waking more than 50 ms late stays at exactly four and exactly one send blocks, so the share of events paying the cost falls from 7.0% to 0.8%. The median is flat at 1.6 ms throughout and the maximum flat at 103.5 ms: the cost is present in every run and dilutes in every run, which is the signature of something paid once. Had it been a per-event constant, the count would have grown with the window and the median would have stayed at 103 ms.

Redis was run in the same sweep on the same events, with the same loop trace, and has *no* events waking more than 50 ms late and *no* blocking send at any window. Its worst single event anywhere is under 25 ms. Instrumenting both arms identically matters here for a reason internal to this paper: an earlier version of this table reported Redis's counts as zero when its producer in fact carried no trace, which is the absence of a measurement dressed as one — exactly the failure the paper is about. We added the trace to the Redis producer and re-ran both arms together.

The mechanism, read directly off the loop trace. The per-event trace shows the same five-event signature in every Kafka run, at every window:

event	wake late (ms)	send blocked (ms)
0	0.17	**102.60**
1	**102.96**	0.07
2	**103.08**	0.05
3	**103.17**	0.04
4	**103.30**	0.05
5	0.94	0.27
6	1.42	0.06

The first `produce()` blocks for 102.6 ms fetching metadata and creating the topic. The replay loop is single-threaded, so every event due while it is blocked wakes roughly 103 ms late and then sends in tens of microseconds. There are exactly four of them, and the reason is the sport rather than the harness: this plan's first five events all carry t_{sim}=0, because a kickoff is recorded as a pass, a ball receipt and a carry inside the same second. That is not particular to this match — every one of the eleven plans in our corpus opens with at least five events at t_{sim}=0. A football feed therefore delivers its densest burst precisely when the producer is coldest, which is the worst possible alignment for a start-up cost and the reason it lands in the median rather than the tail. From event 5 the loop is in steady state at about 1 ms. One cost, paid once, is charged to five events; the sixth onward pays nothing. Redis shows no such prologue because `XADD` to a new stream creates it in the same round trip.

That closes it. Five events carry the cost, seven were matched, and the median of seven values of which at least four are 103 ms *is* 103 ms. The published figure is the arithmetic of its own prologue.

Why the original corpus could not see that, and how we know. The E1 runs behind Table 5 replayed the first 600 s of match time, which in this plan holds **507 events**. They matched a **median of seven** — 745 events across 100 Kafka runs, a match rate near one percent. The window was not the problem; the join was. And the seven that survived it are not a random seven.

That last claim is arithmetic rather than inference. Those runs report a median producer scheduling lag of 102.93 ms over seven values, and a median of seven values at 102.93 ms requires *at least four of the seven* to be 102.93 ms or greater. The loop trace says exactly how many events per run are ever that late: four (Table 7), and always the same four — those due while the first send blocks. The matched set must therefore consist almost entirely of the prologue. The plan's first five events are all stamped at t_{sim}=0, so they are the first candidates a consumer sees and the first a lossy join keeps.

The failure thus compounds: a sample of seven is small, but a sample of seven *selected towards the contaminated events* is worse than small. Redis reports 1.75 ms on the same events because opening a stream with `XADD` does not pay what Kafka's first send pays in metadata fetch and topic creation, so the same selection costs it nothing.

Why not knowing E1's replay rate does not matter here. The blocking first send lasts 102.6 ms of *wall* time, so at a replay rate of R times real time it spans $0.103R$ seconds of *match* time. Evaluating that against the plan for each rate E1 could have run at:

Replay rate	Match time spanned	Events inside the prologue
1×	0.10 s	5
120×	12.3 s	16
1200×	123.1 s	112

E1 matched a median of seven events per run. At every one of the three candidate rates the prologue already contains at least four events — enough to carry the median of seven — so the matched sample's median is the start-up cost, which is what the corpus reports. The conclusion is therefore insensitive to the replay rate; Section 6.5 goes on to recover that rate from the same start-up cost, but the withdrawal does not depend on the recovery.

This also explains the three properties we previously offered as evidence, and each of them is equally consistent with a start-up cost: a per-run constant looks *constant* within a run; it is *concurrency-invariant* because every run pays exactly one regardless of N; and it is *rate-dependent* because accelerated replay packs more events into the window and dilutes it. We read three signatures of a per-event mechanism into measurements that a per-run mechanism explains at least as well.

The general lesson. The integrity check of Section 4.3 does not catch this. Every one of those runs is causally consistent; nothing is negative; the medians are stable to three significant figures across a hundred runs. What went wrong is that a median over seven events was reported as a property of the system rather than of the window, and the tight agreement across runs made it look robust precisely because the artefact is deterministic. A benchmark can be internally consistent, gate-clean, and still measure its own start-up. We therefore add to Section 8.2 the requirement to report events-per-run alongside any latency percentile, since a percentile over single-digit samples is a statement about the harness.

Table 7. The observation-window sweep that separates a per-run cost from a per-event one. Same match, same driver, same broker, N=1, true real-time replay; only the window changes. Both producers are loop-traced, so the counts for the two systems come from the same instrument. Events emitted grow 8.9× across the sweep, but the number of Kafka events waking more than 50 ms late does not move, and exactly one send blocks in every run; Redis has neither at any window. Counts are medians over three replicates from the per-event trace; percentiles come from the per-run summary.

	Window (s)	Events emitted	Events matched	Scheduling lag (ms) median	Scheduling lag (ms) max	Events > 50 ms late	Blocking sends
Kafka	60	57	57	1.56	103.4	4	1
	180	148	148	1.61	103.5	4	1
	600	507	465	1.58	103.5	4	1
Redis	60	57	57	1.50	17.4	0	0
	180	148	148	1.56	20.4	0	0
	600	507	465	1.53	24.8	0	0

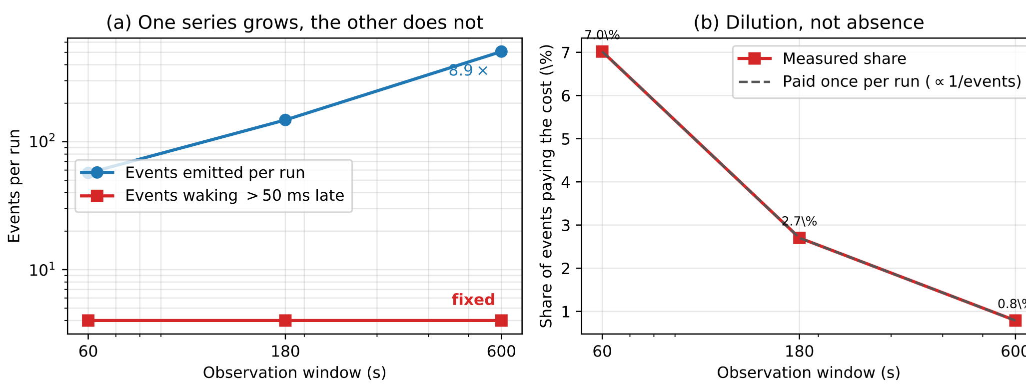

Fig. 4. The window sweep, as a picture. (a) Events emitted per run grow 8.9× while the number waking more than 50 ms late stays at four — a per-event constant would put the two series on parallel lines. (b) The same fact as a proportion: the share of events paying the cost falls as 1/events, the dilution a once-per-run cost predicts. Redis, on the same events, has no affected events at any window.

What we claim instead. On broker transport — the quantity that survived Section 4.3 and is measured over the same events for both systems — Kafka and Redis remain equivalent within one millisecond (Section 7.1). We make no end-to-end claim. The twentyfold figure is withdrawn.

7.3 The model's rules, measured

Section 4.4 derived Pr[inversion] $= F_\Delta(-T_{\text{true}})$ — the effect-size rule itself — and three further consequences, on utilisation, process count and instrument asymmetry. Each was pre-registered with its falsification criterion before the data existed, and all four are tested here on conditions that pass the check.

H1, the effect-size rule. The strongest evidence is the widest contrast, and it is clean: the co-located arm measuring $T_{\text{true}} \approx 0.5$ ms fails wholesale, while the network arm measuring tens of seconds passes every run on the same hardware (Section 6.6). Across the intermediate range, a `netem` delay

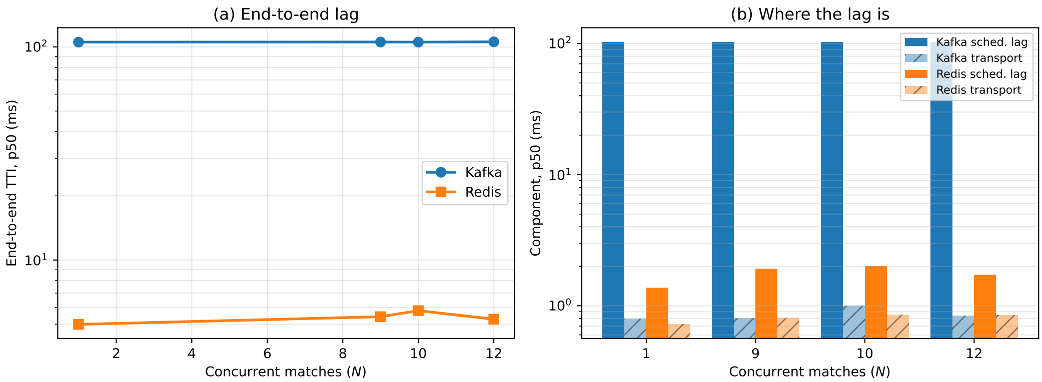

Fig. 5. **Withdrawn.** (a) End-to-end delay against concurrency as originally measured. (b) The decomposition that appeared to place the whole gap in producer scheduling lag. Both panels are retained as evidence for Section 7.2: the runs behind them matched a median of seven events each, so the apparent per-event offset is a per-run start-up cost. Broker transport, the lower series in (b), is unaffected and is what the paper claims.

sweep gives inversion rates 0.076, 0.114, 0.070, 0.034, 0.050 at 0, 1, 5, 20 and 50 ms ($\rho_s = -0.80$, $n = 5$); we report the rank correlation rather than a clean ordering, and we do not lean on its slope, because injected delay also inflates variance and so confounds T_{true} with backlog (Section 8.4). The direction is consistent across both the clean contrast and the confounded sweep. **Supported.**

H2, the utilisation rule. Sweeping background load from idle to oversubscription, with no core pinning so the measured utilisation is the right denominator, inversion rate is flat below half utilisation and then climbs steeply (Table 9). The rank correlation is $\rho_s = 0.98$ ($n = 9$), and the M/G/1 form fits far better than a linear alternative ($R^2 = 0.945$ against 0.640). The predicted knee is present and has the predicted shape. **Supported.**

H4, the oversubscription rule. At constant aggregate event rate, inversion rate rises with the number of concurrent producer/consumer processes: 0.029, 0.082, 0.105, 0.103 at $N = 1, 3, 6, 12$ ($\rho_s = +0.80$, $n = 4$). It is process count, not offered load, that drives the failure. **Supported.**

H3, the asymmetry rule. The model says that what biases a *comparison* is $\mathrm{E}[\Delta]$, and that it is non-zero only when the two endpoints stamp differently. Our two systems do: Redis stamps the acknowledgement on the calling thread the instant `XADD` returns, while Kafka stamps it in a delivery callback that runs on the client's I/O thread and must first be scheduled. So the prediction is specific — making Kafka stamp the same way Redis does should shrink the between-system gap, and shrink it from Kafka's side.

Two earlier attempts did not test this: both compared Kafka's two client libraries, and both of those stamp in callbacks, so the symmetric condition was never created, and we treat them as untested rather than as evidence. Here we add an inline-stamping mode to the Kafka producer that stamps on the calling thread the moment the send resolves — structurally what `XADD` does — and run it against Redis under matched load, both at one in-flight request (Table 8). The gap falls from +0.286 to +0.215 ms, a 25% reduction, and the 0.071 ms it loses comes entirely from Kafka's side: Kafka's median transport drops from 0.392 to 0.322 ms while Redis's holds at 0.106. Removing the callback removed a 0.07 ms systematic offset that was being charged to the broker rather than to the instrument. **Supported.**

Table 8. H3: median broker transport under the two stamping modes, both arms at one in-flight request under matched load. `callback` is the default asymmetric instrument; `inline` stamps Kafka on the calling thread as Redis already does. The between-system gap shrinks, and the shrinkage is entirely on Kafka's side.

Stamp	Kafka (ms)	Redis (ms)	Difference (ms)
`callback` (asymmetric)	0.392	0.106	+0.286
`inline` (symmetric)	0.322	0.107	+0.215

Table 9. H2: inversion rate against achieved CPU utilisation, no core pinning, background load swept from idle to oversubscription. The M/G/1 form fits at $R^2 = 0.945$ against 0.640 for a straight line.

Utilisation ρ	Inversion rate
0.003	0.007
0.253	0.009
0.504	0.022
0.628	0.047
0.753	0.132
0.878	0.207
1.000	0.208
1.000	0.221
1.000	0.264

Table 10. Injected one-way broker delay, applied identically to both systems, five feeds. Medians. The zero-delay row is rejected in both arms and marks the absence of a usable baseline.

Delay	Kafka TTI	Kafka transp.	Redis TTI	Redis transp.
0 ms		*rejected*		
5 ms	12.4 ms	5.6 ms	4,651 ms	4,645 ms
20 ms	77.8 ms	20.8 ms	31,401 ms	31,268 ms
50 ms	336.6 ms	44.9 ms	103,143 ms	102,460 ms

This is the rule with the most direct bearing on the paper's own first result. The spurious effect the audit rejected was a between-system difference; H3 shows that even after the audit, an asymmetric stamp leaves a smaller one behind, in the same direction, on the same pair of systems. An equal instrument is not a nicety.

7.4 The condition that reverses the ordering

Injecting one-way delay identically at both brokers with `tc netem` reverses the ordering completely (Table 10). Kafka tracks the injected delay almost additively. Redis collapses: 4.6 seconds at 5 ms of delay, 31.3 at 20 ms and 102.5 at 50 ms, roughly 900, 1,560 and 2,050 times the delay injected. No run was truncated, so this is amplification, not loss. The Redis arm passes the consistency check on all 15 runs at every delay, for the reason given in Section 6.6.

Magnitudes of this size cannot be a per-event cost: 102 seconds cannot be assembled from 50 ms applied once per event. The account we test is a round-trip-bound consumption pattern. Our Redis consumer acknowledges each entry with `XACK` and waits for the reply — the idiomatic pattern in client tutorials — so a read returning two hundred entries is followed by two hundred sequential round trips. On a fast network this is free: a round trip costs 0.22 ms, a ceiling of ≈4,200 exchanges per second against a demand below one. At 20 ms it has no headroom. The queueing statement is elementary [11]: once service rate falls below arrival rate the queue grows without bound and delay is set by the backlog rather than the network. Kafka's consumer serves from a prefetched buffer and commits on a timer, so it has no per-record round trip to expose.

We stated the falsification criterion in advance: if amortising the acknowledgements does not help, the account is wrong. At one feed under true real-time replay, batching acknowledgements in groups of two hundred takes median TTI from 4,138 ms to 103 ms, a factor of **40.2**, replicated four times with a spread of 4 ms unbatched and 0.4 ms batched. Read-loop instrumentation confirms the mechanism and corrects its shape: each `XREADGROUP` returns a *full* batch, a median of 106 messages, in about 32 ms, so the consumer is not read-bound. The constraint is entirely in the acknowledgement path.

An unexplained null. At five feeds under accelerated replay the same intervention does nothing: 68,960 → 73,598 ms at 20 ms delay and 87,624 → 92,386 ms at 50 ms, both marginally worse batched. We report this prominently because the obvious explanations do not survive inspection. The treatment was applied — the campaign runs a manipulation check and the consumer logs its effective batch size. The measurement is sound — the Redis arm passes on all 15 runs in each cell. The machine is not saturated in general — Kafka in the same runs sits at 78 ms. An earlier version of this work explained the null away as a consistency failure; that explanation was false and we withdraw it. We claim the mechanism at realistic load, where we have both the intervention and the read-loop evidence, and state that its behaviour at five times that load is unexplained.

7.5 Configuration dominates product

A latency comparison that ignores configuration effort misstates the engineering choice.

Each system has exactly one client setting worth one to two orders of magnitude in delay, and in each case the introductory example in the documentation uses the slow value. For Kafka it is producer pipelining: with one outstanding request per connection, which is what a synchronous send example gives, median TTI at one feed was 1,644 ms; with `max.in.flight` at 64 it was 16 ms, a factor of 103. For Redis it is acknowledgement batching: the per-message `XACK` that every tutorial shows costs 4,138 ms behind a 20 ms hop, and batching two hundred at a time costs 103 ms, a factor of 40.2.

Both share the property that makes them hard to catch: they are *free on a co-located testbed.* Neither matters when the broker round trip is 0.2 ms. A benchmark conducted on loopback prices both at zero and will certify as equivalent two clients that differ by two orders of magnitude in deployment. This is the practical corollary of Section 6.6: the conditions that make a testbed convenient are the conditions that make it least informative.

The systems differ in the *shape* of their configuration surface rather than its size. Kafka's latency-relevant settings are numerous, documented together, and mostly safe once pipelining is enabled. Redis's are fewer, but the consequential one is not a setting at all — it is a property of how the application's read loop is written, which no configuration audit will surface.

8 Discussion

8.1 For benchmark authors

Report a physical-consistency audit. Any delay computed as a difference of timestamps taken in different processes admits a sign check, and the check costs nothing. We rejected 58% of our own runs with it, and every rejected run had passed every conventional check we knew to apply.

Better instruments exist [30]; our point is orthogonal to instrument quality — whatever instrument is used, its output should be checked against causality and the retention rate published.

Count your events before you quote a percentile. Our second withdrawal (Section 7.2) came from reporting a median over seven events per run as a property of the system. It survived a hundred runs, a causal-consistency audit, and our own scepticism, because a deterministic per-run artefact reproduces beautifully. Sample size per run belongs next to every percentile, and a benchmark whose window admits single-digit event counts is measuring its own start-up.

Measure at a realistic round-trip time, and treat interventions as interventions. Both configuration defects in Section 7.5 are invisible on loopback, and the one intervention we ran without a manipulation check produced a clean false null.

8.2 Applying the check to another benchmark

The check is worth reporting only if it transfers, so we state it as a procedure rather than as a description of what we happened to do. It applies to any latency computed as a difference of timestamps taken in different processes, threads or hosts — which covers most end-to-end measurements in this literature.

(1) **Identify every span whose endpoints are stamped by different execution contexts.** In our pipeline exactly one of the three components qualifies: scheduling lag and the output interval are stamped within a single process, broker transport is not. Spans stamped within one context cannot violate causality and need no check.
(2) **Record the raw per-event values, not summaries.** The failure is invisible in aggregate — that is its defining property — so a pipeline that writes only percentiles has already discarded the evidence. This is the one requirement that may demand a change to an existing harness.
(3) **Count the fraction of events in which the span is negative**, per run and per component.
(4) **Reject by a stated rule, before looking at results.** We reject a run when any component exceeds 1% inverted or has a negative median. The threshold is a judgement and should be reported with its sensitivity (Section 6.3); what matters more is that it is fixed in advance and applied to every condition, including those whose results look reasonable. Applying it only to suspicious results reproduces the error it exists to prevent.
(5) **Report the retention rate alongside the results**, the way a survey reports its response rate. A benchmark that discards nothing has either an unusually sound instrument or an unexamined one, and a reader cannot tell which without the number.
(6) **Report events-per-run alongside every latency percentile.** This is the check that would have caught our second withdrawal (Section 7.2) and the sign check would not: a median over seven events is a statement about the harness, not the system, and it looks its most convincing when the artefact is deterministic enough to repeat across a hundred runs. Any percentile computed over single-digit samples should be labelled as such.
(7) **Record the achieved rate, per run, next to the results — not the flag that was meant to produce it.** A replay harness has a nominal rate and an achieved one, and they part company silently. Ours parted company by a factor of 120 because the plans carried a compression the flag did not know about (Section 6.5). Derive the setting from the workload rather than assuming it, verify the achieved rate against elapsed wall time before the campaign rather than after, and write the verified figure into an artefact that outlives the raw data. We had to reconstruct the rate of our own earliest corpus by arithmetic on its results, which happened to be possible and will not always be.
(8) **Before attributing an offset to every event, vary the run length and count.** Lengthen the observation window by a known factor and count how many events exceed the offset,

rather than re-computing the average. A per-run cost holds that count fixed while the run grows; a per-event one grows it in proportion. The two are indistinguishable in any percentile, which is why we report the count. In our sweep the count stayed at four while events per run grew several times over (Table 7).

(9) **Where retention differs between arms, bound the selection it introduces.** Unequal rejection is itself a bias, and Section 6.4 gives the worst-case imputation we use.

The cost is one pass over data the harness already produces. In our implementation the rule is under two hundred lines and runs in seconds over 2,266 runs, which is why we argue it belongs in routine practice rather than in a dedicated study.

When it matters most. Equation 2 tells a reader where to be careful without repeating our experiments: risk rises as the measured quantity approaches the instrument's own scheduling jitter, and as utilisation approaches saturation. A benchmark measuring milliseconds on a loaded machine is in the dangerous region; one measuring seconds, or running well below saturation, is not. That is a rule of thumb, and Section 8.5 says what would be needed to make it quantitative.

8.3 For practitioners

The brokers themselves are equivalent within a millisecond for any workload resembling this one, and neither degrades across its concurrency range. Redis does hold a real, reproducible transport advantage — about 0.4 ms per operation (Section 7.1), the direction the grey literature reports — but at four parts in 100,000 of a seconds-scale end-to-end budget it cannot be the basis for a choice. Selection should rest on durability semantics, operational familiarity and cost — not on a sub-millisecond transport gap, and not on published latency benchmarks conducted at rates the deployment will never see.

Where a consumer sits across a network, audit the *consumption pattern* before choosing a product. A client that acknowledges per message and waits for the reply converts ordinary network delay into seconds of staleness; batching removes it. Kafka is not immune by virtue of being Kafka, but because its client already amortises.

One practical asymmetry survives our withdrawal, and it is a start-up property rather than a steady-state one. A Kafka producer's first `produce()` blocks for roughly 100 ms while it fetches metadata and creates the topic (Section 7.2); Redis's `XADD` creates the stream in the same round trip and blocks for under a millisecond. On a long-lived producer this is paid once and is irrelevant. It matters in two situations: where producers are short-lived — one per match, per request, or per serverless invocation — and where the events that matter most arrive first, as a kickoff burst does. In both, pre-create the topic and issue a warm-up send before the feed opens.

8.4 Threats to validity

Construct validity: the check may not measure what we claim. A negative span is proof that *something* is wrong, but our attribution of it to scheduler delay in reaching the clock is an inference. Two rival explanations are available. Clock skew is ruled out directly for Testbed A, where both processes read the same operating-system clock; it is not ruled out on Testbed B, where producer and consumer may sit on different hosts, and there we rely on the co-located conditions failing at rates comparable to the distributed ones.

Coarse kernel timestamp granularity would also produce inversions, and that rival makes a different prediction we can test. Quantisation acts on each event independently, so its inversions would be scattered at random through a run; a descheduling event, by contrast, makes a *consecutive run* of events wake late together. A Wald–Wolfowitz runs test on the sequence of inversion signs separates the two: independence gives $z \approx 0$, clustering $z \ll 0$. Across conditions the inversions are strongly clustered — median $z = -6.9$ at the co-located floor, −6.8 idle, −5.1 saturated — and the clustering is present even with no background load, so it is intrinsic to the stamping path rather than induced by contention. This is the signature of a shared scheduling delay, not of independent quantisation, and Section 4.1 additionally reports the measured timer granularity on both platforms. We regard the Testbed A attribution as well supported and the Testbed B attribution as probable rather than established.

Internal validity: the audit selects. The check rejects unequally between arms, so the surviving corpus is not a random sample of the measured one. This is the most serious internal threat, and Section 6.4 addresses it with a worst-case bound rather than an argument. That bound holds only while retention exceeds one half, and our tightest cell sits 2.8 points above that line; a slightly worse campaign would have left the equivalence claim undefendable.

Internal validity: the surviving comparison is small. After rejection, per-cell samples run from 8 to 35. The N=1 cell cannot support inference and we treat it as descriptive; conclusions rest on the levels with 19 or more runs.

External validity: one workload, two systems, two platforms. We demonstrate the failure on a sparse bursty feed, two brokers and two operating systems. The model of Section 4.4 predicts it wherever cross-process spans are measured near the instrument's own jitter under load, but we have not shown that. Sharma et al. [25] report the same symptom in an unrelated domain, which is suggestive but is not our evidence.

Construct validity: a percentile is not automatically a property of the system. The 103 ms producer offset reproduced across both testbeds, four concurrency levels and 164 runs, and we took that breadth as evidence it was a property of the system. It was a property of the window (Section 7.2). Reproducibility across runs distinguishes a deterministic effect from a noisy one; it does not distinguish a system effect from a harness effect, and we treated it as though it did. Every percentile in this paper is now reported with the events per run behind it.

Construct validity: the injected-delay sweep is not a clean effect-size manipulation. H1's quantitative slope (Section 7.3) is estimated from a `tc netem` delay sweep, on the assumption that injecting d ms of delay simply raises T_{true} by d. It does not: on our bursty feed, a per-delivery delay builds a queue, so the measured transport *variance* climbs from 10 ms^2 at zero delay to 9,200 ms^2 at 50 ms — a clean offset would leave it unchanged. The delay sweep therefore confounds T_{true} with backlog-driven jitter, and we do not lean on its fitted slope. H1's robust evidence is the comparison it does not confound: the co-located arm, where $T_{\text{true}} \approx 0.5$ ms, fails wholesale, while the network arm, where T_{true} is tens of seconds, passes 15/15 on the same hardware (Section 6.6). Testing the fine-grained slope cleanly would need a constant-offset instrument that does not perturb the queue — a run we flag as future work.

Conclusion validity: multiplicity and pre-specification. Holm families are declared over the design actually executed rather than chosen afterwards, and equivalence margins were fixed before testing. The threshold in the check was fixed before the final campaign but *after* earlier campaigns had run, which is weaker than pre-registration; the sensitivity analysis of Section 6.3 exists partly to compensate.

8.5 Limitations

The surviving evidence is narrower than the design. The audit removed the throughput sweep, the connection sweep above ten, the cluster arm, the durability quantification and all of Testbed A. We cannot characterise behaviour at the concurrency a multi-tenant platform would see.

E1's replay rate is inferred, not documented. Section 6.5 recovers it from the data and the recovery is, we think, sound, but it is an inference from a 52.34 ms median in one cell rather than a setting anyone recorded. A reader who does not accept the inference should read Section 7.1 as establishing that both arms met the same rate, which is all the between-system comparison needs.

A small replication at a verified rate does not match E1's transport figures. Three $N=1$ runs give Kafka transport 0.45 ms against Redis's 0.09 ms — a fivefold gap where E1 reports near-equality — with both systems below their E1 values. Three runs settle nothing and the conditions differ in window length, so we draw no conclusion from it; but we record it rather than omit it, because it is the one observation we have that points away from Section 7.1, and a properly powered replication is the first thing this work needs next.

The mechanism is established for our client, not for Kafka in general. The blocking first `produce()` is visible in the loop trace of every run (Section 7.2), so for this client the attribution is direct rather than inferred. Whether a different client, or a pre-created topic, or a broker configured against auto-creation would pay the same cost, we did not test. The practitioner advice in Section 8 is written to be safe either way.

H3 is measured at one operating point. The symmetric-stamping comparison (Section 7.3) was run at one in-flight request and one load level, because inline stamping requires the single in-flight setting. The direction and the one-sidedness of the effect are what the model predicts, but we do not characterise how the 0.07 ms offset scales with load or pipelining.

The five-feed acknowledgement null is unexplained, as set out in Section 7.4.

One workload. The arrival-rate result should generalise to any sparse bursty feed, but we demonstrate it on one.

9 Conclusion

We set out to compare two message brokers for a real-time sports feed under varying concurrency. We obtained a complete answer, and it was physically impossible.

The audit that found it is this paper's principal contribution. A negative broker delay is proof of instrument failure, and checking for it rejected 1,321 of 2,266 runs, including every run behind a large, significant, theory-confirming result. The rejected corpus had passed plausible medians, narrow intervals, large effect sizes, the architecturally expected direction, and six prior rounds of correction. It failed only against arithmetic. The property that makes the check general is that, being a test against zero, it bites hardest exactly where the measured effect is smallest — which is to say on the delicate comparisons that benchmark papers exist to make.

A second headline then failed the same way, and we withdraw it too. A twentyfold end-to-end gap between the systems, which we had attributed to client behaviour and built a recommendation on, turned out to be a per-run start-up cost read as a per-event constant: the runs behind it matched a median of seven events each. The integrity check does not catch that one. Causal consistency is necessary and not sufficient, and a percentile computed over single-digit samples describes the harness rather than the system — the more so because a deterministic artefact reproduces across a hundred runs and looks robust for exactly the wrong reason.

What survives is narrower and better evidenced. The brokers are equivalent within a millisecond and neither degrades with concurrency, robustly to the audit's own selection effect — though a powered replication resolves inside that margin a real 0.41 ms advantage for Redis that the original seven-event corpus reported as a tie. Each system has one client setting worth one to two orders of magnitude, both free on the co-located testbeds where such settings are normally evaluated. And the model of the failure now carries measured rules rather than a derivation: inversion probability falls as the measured quantity grows, rises with concurrent process count, and follows M/G/1 waiting time in utilisation with the predicted knee; and a symmetric instrument removes a quarter of the residual between-system difference, the same asymmetry that manufactured our first result in the large. The check binds hardest where the effect is smallest, and we can now say so with a curve rather than an anecdote.

One smaller episode makes the same point from the other side. Checking our own provenance, we found that no surviving artefact recorded the replay rate of the earliest corpus, and that the records we did have disagreed. We recovered it anyway — the start-up cost is a wall-clock constant, so the number of events it swallows encodes the rate, and one cell of fifteen runs reading 52.34 ms where both modes are 1.6 and 103.5 fixes it at four late events and hence at true real time. That is the same instrument as the sign check: a physical constraint deciding what the bookkeeping could not. It is also luck, and no basis for a method. The rule is to write the number down.

The original question turned out to be the least interesting thing we could have asked. That is worth stating plainly, because the path from it to the results above ran entirely through measurements we had every reason to trust — twice.

Artefact Availability

All code, replay plans, aggregated results and analysis are available at https://github.com/Gustolandia/streaming-latency-sports. Event data is the StatsBomb open dataset under CC BY-NC 4.0; raw files are not redistributed but are re-fetchable from a pinned upstream commit. Every run records its git commit and per-file SHA-256. A regression suite recomputes every figure in this paper from committed data and fails if paper and data disagree.

References

[1] Arvind Arasu, Mitch Cherniack, Eduardo Galvez, David Maier, Anurag S. Maskey, Esther Ryvkina, Michael Stonebraker, and Richard Tibbetts. 2004. Linear Road: A Stream Data Management Benchmark. In *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*. 480–491.

[2] Ashok Chandrasekar and Jason Kramberger. 2026. Identifying and mitigating systemic measurement bias in production LLM inference benchmarks. *arXiv preprint arXiv:2605.24217* (2026).

[3] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, et al. 2013. Spanner: Google's globally distributed database. *ACM Transactions on Computer Systems* 31, 3 (2013), 1–22.

[4] ThreadSafe Diaries. 2025. I Benchmarked Kafka, RabbitMQ, and Redis Streams, The Winner Surprised Me. Accessed: June 15, 2026. https://medium.com/@ThreadSafeDiaries/i-benchmarked-kafka-rabbitmq-and-redis-streams-the-winner-surprised-me-cf3f484eb7b2

[5] Bradley Efron. 1979. Bootstrap methods: another look at the jackknife. *The Annals of Statistics* 7, 1 (1979), 1–26.

[6] Guenter Hesse, Christoph Matthies, Michael Perscheid, Matthias Uflacker, and Hasso Plattner. 2021. ESPBench: The Enterprise Stream Processing Benchmark. In *Proceedings of the ACM/SPEC International Conference on Performance Engineering*. 201–212.

[7] J. L. Hodges and E. L. Lehmann. 1963. Estimates of location based on rank tests. *The Annals of Mathematical Statistics* 34, 2 (1963), 598–611.

[8] Sture Holm. 1979. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.

[9] Raj Jain. 1991. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling.* Wiley.

[10] JusDB. 2025. Redis Streams vs Kafka: Event Streaming Architecture Comparison 2025. Accessed: June 15, 2026. https://www.jusdb.com/blog/redis-streams-vs-kafka-event-streaming-comparison-2025

[11] Leonard Kleinrock. 1975. *Queueing Systems: Volume I - Theory.* Wiley-Interscience.

[12] Jay Kreps, Neha Narkhede, and Jun Rao. 2011. Kafka: a distributed messaging system for log processing. *Proceedings of the 6th ACM Symposium on Networked Systems Design and Implementation* (2011).

[13] William H. Kruskal and W. Allen Wallis. 1952. Use of ranks in one-criterion variance analysis. *J. Amer. Statist. Assoc.* 47, 260 (1952), 583–621.

[14] Daniël Lakens. 2017. Equivalence tests: a practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science* 8, 4 (2017), 355–362.

[15] Leslie Lamport. 1978. Time, clocks, and the ordering of events in a distributed system. *Commun. ACM* 21, 7 (1978), 558–565.

[16] Hongyou Liu, Will Hopkins, A. Miguel Gomez, and J. Sampaio Molinuevo. 2013. Inter-operator reliability of live football match statistics from OPTA Sportsdata. *International Journal of Performance Analysis in Sport* 13, 3 (2013), 803–821.

[17] Henry B. Mann and Donald R. Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics* 18, 1 (1947), 50–60.

[18] David L. Mills. 1991. Internet time synchronization: the network time protocol. *IEEE Transactions on Communications* 39, 10 (1991), 1482–1493.

[19] Muzeeb Mohammad. 2025. Analysis of design patterns and benchmark practices in Apache Kafka event-streaming systems. *arXiv preprint arXiv:2512.16146* (2025).

[20] OpenMessaging Project, Linux Foundation. 2018. The OpenMessaging Benchmark Framework. Open standard, multi-platform messaging performance benchmark. Available at openmessaging.cloud/docs/benchmarks/.

[21] Luca Pappalardo, Paolo Cintia, Alessio Rossi, Emanuele Massucco, Paolo Ferragina, Dino Pedreschi, and Fosca Giannotti. 2019. A public data set of spatio-temporal match events in soccer competitions. *Scientific Data* 6, 1 (2019), 236.

[22] Salvatore Sanfilippo. 2017. Redis streams: a new data type for real-time streaming. *Redis Labs Blog* (2017). https://redis.io/topics/streams-intro

[23] Bianca Schroeder, Adam Wierman, and Mor Harchol-Balter. 2006. Open versus closed: a cautionary tale. In *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI).* 239–252.

[24] Donald J. Schuirmann. 1987. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics* 15, 6 (1987), 657–680.

[25] Ankur Sharma, Deep Shah, David Lariviere, and Hesham ElBakoury. 2026. Time, causality, and observability failures in distributed AI inference systems. *arXiv preprint arXiv:2604.21361* (2026). Open Compute Project, Unified Intelligent Infrastructure workstream.

[26] Anshu Shukla, Shilpa Chaturvedi, and Yogesh Simmhan. 2017. RIoTBench: An IoT benchmark for distributed stream processing systems. *Concurrency and Computation: Practice and Experience* 29, 21 (2017), e4257.

[27] StatsBomb. 2023. StatsBomb Open Data. *GitHub Repository* (2023). https://github.com/statsbomb/open-data

[28] Michael Stonebraker, Ugur Cetintemel, and Stan Zdonik. 2005. The 8 requirements of real-time stream processing. *ACM SIGMOD Record* 34, 4 (2005), 42–47.

[29] Pete Tucker, Kristin Tufte, Vassilis Papadimos, and David Maier. 2008. *NEXMark: A Benchmark for Queries over Data Streams.* Technical Report. OGI School of Science and Engineering, Oregon Health and Science University.

[30] Shinhyung Yang, Jiun Jeong, Bernhard Scholz, and Bernd Burgstaller. 2022. Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads. *arXiv preprint arXiv:2205.09325* (2022).

[31] Praneeth Yerrapragada. 2025. kafka-bullmq-benchmark: A comprehensive distributed systems performance comparison. Includes white paper with methodology; Accessed: June 15, 2026. https://github.com/praneethys/kafka-bullmq-benchmark